

1. What is SQL, and why is it essential in database management?

SQL (Structured Query Language) is a standard language used to interact with relational databases. It allows users to create, insert, update, delete, and retrieve data efficiently. SQL is essential because it organizes and manages large amounts of data in a structured way. It ensures consistency, integrity, and security of the data. Almost all modern database systems support SQL as their primary language.

2. Explain the difference between DBMS and RDBMS.

A DBMS (Database Management System) stores and manages data, but it does not maintain relationships between tables. An RDBMS (Relational Database Management System) stores data in tables and establishes relationships using primary and foreign keys. DBMS is suited for smaller datasets, while RDBMS handles large, complex, and relational data. Examples of RDBMS are MySQL, Oracle, and SQL Server.

3. Describe the role of SQL in managing relational databases.

SQL acts as an interface between users and relational databases. It allows defining database structures using DDL, manipulating data with DML, and retrieving data with queries (DQL). SQL also provides transaction management (TCL) to maintain consistency and security control (DCL) for user access. Without SQL, managing relational databases would be very difficult and inefficient.

4. What are the key features of SQL?

SQL is a simple and standardized language widely used across database systems. It provides commands for data definition, manipulation, and retrieval. SQL supports powerful queries with filtering, sorting, and joins across multiple tables. It maintains data integrity using constraints and offers transaction management for reliability. Security is ensured through access control features like GRANT and REVOKE.

5. What are the basic components of SQL syntax?

SQL syntax is made up of keywords, identifiers, operators, and clauses. Keywords like SELECT, INSERT, UPDATE, and DELETE define the action to be performed. Identifiers include table names, column names, and aliases. Operators (=, <, >, AND, OR) are used for conditions. Clauses like WHERE, GROUP BY, and ORDER BY refine the query. Together, these components form valid SQL statements.

6. Write the general structure of an SQL SELECT statement.

SELECT column1, column2, ...

FROM table_name

WHERE condition

GROUP BY column

HAVING condition

ORDER BY column ASC|DESC;

7. Explain the role of clauses in SQL statements.

Clauses define the scope and behavior of an SQL statement. For example, WHERE filters rows, GROUP BY groups data, HAVING applies conditions on groups, and ORDER BY sorts the output. Clauses make SQL statements flexible and powerful by refining data selection and manipulation. Without clauses, SQL queries would return all data without control or organization.

8. What are constraints in SQL? List and explain the different types of constraints.

Constraints in SQL are rules applied to table columns to ensure data accuracy and integrity.

- NOT NULL: Column cannot have NULL values.
- UNIQUE: Ensures all values in a column are distinct.
- PRIMARY KEY: Combination of NOT NULL and UNIQUE; uniquely identifies each row.
- FOREIGN KEY: Links two tables and enforces referential integrity.
- CHECK: Ensures values satisfy a specific condition.
- DEFAULT: Assigns a default value if none is provided.

9. How do PRIMARY KEY and FOREIGN KEY constraints differ?

☐ A PRIMARY KEY uniquely identifies each record in a table. It does not allow NULL or duplicate values. Each table can have only one primary key.

☐ A FOREIGN KEY is used to create a link between two tables. It refers to the primary key in another table, ensuring that relationships between tables remain valid.

? Primary key = uniqueness within the table, Foreign key = relationship across tables.

10. What is the role of NOT NULL and UNIQUE constraints?

- NOT NULL: Ensures a column must always have a value (no empty entries).
Example: emp_name should never be NULL.
- UNIQUE: Ensures all values in a column are distinct (no duplicates allowed).
Example: email in a user table must be unique.
- Together, they maintain data validity by avoiding missing and duplicate values in important fields.

11. Define the SQL Data Definition Language (DDL).

DDL (Data Definition Language) is a part of SQL used to define and manage the structure of database objects like tables, views, and indexes. It includes commands such as CREATE, ALTER, DROP, and TRUNCATE. These commands work at the schema level, not on individual rows of data. DDL statements are auto-committed, meaning changes are permanent once executed.

12. Explain the CREATE command and its syntax.

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    ...  
);
```

13. What is the purpose of specifying data types and constraints during table creation?

Data types define what kind of data can be stored in a column (e.g., INT, VARCHAR, DATE). This ensures memory efficiency and prevents invalid data entry. Constraints (like NOT NULL, UNIQUE, PRIMARY KEY) enforce rules on the data to maintain accuracy and integrity. Together, data types and constraints help keep the database reliable, consistent, and error-free.

14. What is the use of the ALTER command in SQL?

The ALTER command in SQL is used to modify the structure of an existing table without deleting its data. It allows you to add, modify, or drop columns and constraints. You can also rename columns or change their data type. In short, ALTER helps keep the database schema flexible and adaptable as requirements change.

15. How can you add, modify, and drop columns from a table using ALTER

- Add a new column:

```
ALTER TABLE employees
```

```
ADD age INT;
```

- Modify a column (data type/size):

```
ALTER TABLE employees
```

```
MODIFY emp_name VARCHAR(100);
```

- Drop (remove) a column:

```
ALTER TABLE employees
```

```
DROP COLUMN age;
```

16.What is the function of the DROP command in SQL?

The DROP command in SQL is used to permanently delete database objects such as tables, views, or databases. When a table is dropped, its structure, all rows of data, and any dependent objects like indexes are removed. It is different from DELETE or TRUNCATE because those only remove data but keep the table structure. Once executed, DROP cannot be rolled back unless a backup exists.

17. What are the implications of dropping a table from a database?

Dropping a table removes it completely along with all stored data, structure, and constraints. Any foreign key relationships pointing to that table will also break. Users and applications relying on the table will face errors if they query it after dropping. Therefore, dropping should be done with caution, usually after ensuring backups or confirming the table is no longer required.

18. Define the INSERT, UPDATE, and DELETE commands in SQL.

❓ INSERT: Adds new rows of data into a table. Example:

```
INSERT INTO employees(emp_id, emp_name) VALUES (101, 'Raj');
```

❓ UPDATE: Modifies existing data in one or more rows of a table. Example:

```
UPDATE employees SET emp_name = 'Ravi' WHERE emp_id = 101;
```

❓ DELETE: Removes rows of data from a table. Example:

```
DELETE FROM employees WHERE emp_id = 101;
```

19. What is the importance of the WHERE clause in UPDATE and DELETE operations?

The WHERE clause specifies which rows should be updated or deleted. Without WHERE, the command applies to all rows in the table, which may cause accidental data loss. For example:

```
DELETE FROM employees; -- removes all records
```

```
DELETE FROM employees WHERE emp_id = 101; -- removes only one record
```

20. What is the SELECT statement, and how is it used to query data?

The SELECT statement is used to retrieve data from one or more tables in a database. It allows you to specify the columns to fetch, apply conditions, and format the output.

Example:

```
SELECT emp_name, salary
```

```
FROM employees;
```

This query returns the emp_name and salary of all employees. SELECT is the main command in SQL for querying data.

21. Explain the use of the ORDER BY and WHERE clauses in SQL queries.

- WHERE clause: Filters rows based on specified conditions. Example:

```
SELECT * FROM employees WHERE salary > 50000;
```

- ORDER BY clause: Sorts the query result in ascending (ASC) or descending (DESC) order. Example:

SELECT emp_name, salary FROM employees ORDER BY salary DESC;

Together, WHERE and ORDER BY make queries more precise and organized.

22. What is the purpose of GRANT and REVOKE in SQL?

- GRANT is used to give specific privileges (like SELECT, INSERT, UPDATE, DELETE) to a user or role on database objects.
- REVOKE is used to remove previously granted privileges from a user or role.
- These commands help control who can access or modify data in a database, ensuring security and proper access management.

23. How do you manage privileges using these commands?

- To give privileges:

GRANT SELECT, INSERT ON employees TO user1;

- To remove privileges:

REVOKE INSERT ON employees FROM user1;

- By carefully granting and revoking privileges, a database administrator can manage user access, prevent unauthorized changes, and maintain data security.

24. What is the purpose of the COMMIT and ROLLBACK commands in SQL?

❓ COMMIT: Saves all changes made by the current transaction permanently to the database.

❓ ROLLBACK: Undoes all changes made in the current transaction, restoring the database to its previous state.

❓ These commands ensure data consistency and control when making multiple changes to the database

25. Explain how transactions are managed in SQL databases.

- ❑ A transaction is a sequence of one or more SQL operations treated as a single unit.
- ❑ Transactions follow the ACID properties: Atomicity, Consistency, Isolation, Durability.
- ❑ SQL uses COMMIT to finalize a transaction and ROLLBACK to undo it if an error occurs.
- ❑ Optional SAVEPOINTS can be set to roll back to a specific point within a transaction.

26. Explain the concept of JOIN in SQL. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

- ❑ A JOIN in SQL is used to combine rows from two or more tables based on a related column (usually a primary key and foreign key).
- ❑ INNER JOIN: Returns only rows with matching values in both tables.
- ❑ LEFT JOIN: Returns all rows from the left table, and matching rows from the right table (NULL if no match).
- ❑ RIGHT JOIN: Returns all rows from the right table, and matching rows from the left table (NULL if no match).
- ❑ FULL OUTER JOIN: Returns all rows from both tables, with NULLs where there is no match.

27. How are joins used to combine data from multiple tables?

- Joins allow queries to retrieve related data from different tables in a single result set.
- Example: To display employees with their department names:

```
SELECT e.emp_name, d.depart_name  
FROM employees e  
INNER JOIN departments d  
ON e.depart_id = d.depart_id;
```

28. What is the GROUP BY clause in SQL? How is it used with aggregate functions?

- The GROUP BY clause is used to group rows that have the same values in one or more columns.
- It is commonly used with aggregate functions like COUNT(), SUM(), AVG(), MAX(), and MIN().
- Example: Count the number of employees in each department:

```
SELECT depart_id, COUNT(emp_id) AS total_employees  
FROM employees  
GROUP BY depart_id;
```

29. Explain the difference between GROUP BY and ORDER BY.

- GROUP BY: Groups rows based on column values for aggregation; does not sort data by default.
- ORDER BY: Sorts the query result in ascending (ASC) or descending (DESC) order; does not group data.
- Both clauses can be used together: first GROUP BY to aggregate, then ORDER BY to sort the aggregated results.

30. What is a stored procedure in SQL, and how does it differ from a standard SQL query?

❓ A stored procedure is a precompiled collection of SQL statements stored in the database and executed as a single unit.

❓ Unlike a standard SQL query, it can accept parameters, include conditional logic, loops, and handle multiple operations at once.

❓ Stored procedures improve reusability, consistency, and performance compared to running individual queries repeatedly.

31. Explain the advantages of using stored procedures.

- Reusability: Can be called multiple times without rewriting code.
- Performance: Precompiled and optimized by the database engine.
- Security: Users can execute procedures without direct access to underlying tables.
- Maintainability: Changes can be made in the procedure without modifying application code.
- Reduced network traffic: Multiple SQL statements execute on the server in one call.

32. What is a view in SQL, and how is it different from a table?

- A view is a virtual table based on the result of a SQL query. It does not store data physically but shows data from one or more tables.
- Unlike a table, a view cannot have its own data; it reflects changes made in the underlying tables.
- Example:

```
CREATE VIEW EmployeeView AS
```

```
SELECT emp_name, salary FROM employees WHERE salary > 50000;
```

33. Explain the advantages of using views in SQL databases

- Simplifies complex queries: Users can access pre-defined queries as a single object.
- Security: Restricts access to sensitive columns or rows.
- Data abstraction: Hides underlying table structure from users.
- Consistency: Ensures uniform representation of data across applications.
- Easy maintenance: Changes in underlying tables automatically reflect in the view.

34. What is a trigger in SQL? Describe its types and when they are used.

- A trigger is a special stored procedure that automatically executes in response to certain events on a table, such as INSERT, UPDATE, or DELETE.
- Types of triggers:
 - BEFORE Trigger: Executes before the specified action.
 - AFTER Trigger: Executes after the specified action.
 - INSTEAD OF Trigger: Used on views to execute instead of the triggering action.
- Triggers are used for data validation, enforcing business rules, auditing changes, and maintaining consistency automatically.

35. Explain the difference between INSERT, UPDATE, and DELETE triggers

- INSERT Trigger: Fires when a new row is added to a table. Example: Log every new employee added.
- UPDATE Trigger: Fires when existing row data is modified. Example: Track salary changes in an audit table.
- DELETE Trigger: Fires when a row is deleted. Example: Keep a backup record before deletion.
- Each trigger type reacts to a specific DML operation to automate database tasks.

36. What is PL/SQL, and how does it extend SQL's capabilities?

❓ PL/SQL (Procedural Language/SQL) is Oracle's procedural extension of SQL.

❓ It combines SQL's data manipulation capabilities with procedural constructs like loops, conditions, and variables.

❓ PL/SQL allows writing programs, blocks, functions, and procedures that can handle complex logic, which standard SQL alone cannot do.

❓ Example: Using loops to process multiple rows or conditional statements to apply business rules.

37. List and explain the benefits of using PL/SQL

- ❓ Block structure: Organizes code into reusable, logical blocks.
- ❓ Performance: Executes multiple SQL statements on the server, reducing network traffic.
- ❓ Error handling: Built-in exception handling for runtime errors.
- ❓ Reusability: Functions, procedures, and packages can be reused across applications.
- ❓ Security: Encapsulates logic and controls access to underlying data.
- ❓ Integration: Works seamlessly with SQL for complex database operations.

38. What are control structures in PL/SQL? Explain the IF-THEN and LOOP control structures.

- Control structures in PL/SQL are used to control the flow of execution of statements based on conditions or repetitions.
- IF-THEN: Executes a block of code only if a specified condition is true.

IF salary > 50000 THEN

DBMS_OUTPUT.PUT_LINE('High salary');

END IF;

- LOOP: Repeats a block of code multiple times until a condition is met.

LOOP

counter := counter + 1;

EXIT WHEN counter > 5;

END LOOP;

39. How do control structures in PL/SQL help in writing complex queries?

- ❓ They allow conditional execution and iteration, enabling logic that SQL alone cannot handle.
- ❓ With IF-THEN and LOOP, you can process multiple rows, validate data, and apply business rules programmatically.

☐ Control structures make PL/SQL flexible, powerful, and suitable for complex database applications.

40. What is a cursor in PL/SQL? Explain the difference between implicit and explicit cursors.

☐ A cursor is a pointer that allows PL/SQL to process multiple rows returned by a query one at a time.

☐ Implicit cursor: Automatically created by PL/SQL for single-row queries (SELECT INTO). You don't declare it explicitly.

☐ Explicit cursor: Declared by the programmer to handle multi-row queries. You must open, fetch, and close it explicitly.

41. When would you use an explicit cursor over an implicit one

- Use an explicit cursor when a query returns multiple rows and you want to process them one by one in a loop.
- Implicit cursors are sufficient for single-row queries, but explicit cursors provide more control over multi-row processing, like fetching, looping, and row-by-row manipulation.

42. Explain the concept of SAVEPOINT in transaction management. How do ROLLBACK and COMMIT interact with savepoints?

- A SAVEPOINT is a marker within a transaction that allows partial rollback of changes.
- You can set multiple savepoints in a transaction to undo only certain steps without affecting the entire transaction.
- ROLLBACK TO savepoint_name undoes all changes after the savepoint, but keeps earlier changes intact.
- COMMIT makes all changes, including those before savepoints, permanent; savepoints are then cleared.

43. When is it useful to use savepoints in a database transaction?

☐ Savepoints are useful in complex transactions involving multiple operations.

☐ They allow partial undo if an error occurs in the middle of a transaction.

❓ This provides better control, reduces data loss, and maintains consistency without rolling back the entire transaction.